Q0 Before you start; before you submit

I read Handout 5 on Turing reductions before starting, and the reductions below follow it: name the oracle, give pseudocode for the machine that uses it, show that the machine always halts, and prove correctness in both directions. I did not use Rice's Theorem anywhere. I did not collaborate with anyone on this homework.

Q1 Self assessment

The part of HW5 with the most room for improvement is Question 5(a), the turingmachine.io machine that adds 2 to a binary number. Here is the code I submitted:

```
# Adds 2 to a binary number.
input: '1011'
blank: ' '
start state: right
table:
  # scan to the rightmost digit
  right:
    [1,0]: R
    ' '  : {L: skip}
  # step over the least significant bit (it does not change)
  skip:
    [1,0]: {L: carry}
  # then carry the 1 into the remaining bits
  carry:
    1      : {write: 0, L}
    [0,' ']: {write: 1, L: done}
  done:
```

What was deficient: the machine is wrong on the empty input, which represents the number zero. On a blank tape, state `right` immediately reads a blank and moves left into `skip`. But `skip` only has rules for `1` and `0`, so the machine gets stuck on a blank with no matching transition. The tape is still empty, so the output is 0 instead of $0 + 2 = 2 = 10_2$. I only tested on inputs like `1011` and `1`, where there is a least significant bit to step over, and never asked what happens when there is no bit at all. The posted solution handles this case with a separate `empty` state, and the rubric gives 4 of the 7 points for a fully correct machine, so this edge case matters.

How to fix it: the first cell has to be inspected before the scan starts, because that is the only moment the machine can tell an empty tape from the end of a nonempty one. So I would add a new start state that looks at the first cell: on a digit it moves right into the usual scan, and on a blank it goes to a state that writes the `0` of the answer `10` and lets the carry logic write the leading `1`. In turingmachine.io syntax:

```
start state: start
  start:
    [1,0]: {R: right}
    ' '  : {R: empty}
  empty:
    [1,0,' ']: {write: 0, L: carry}
```

with `right`, `skip`, `carry`, `done` unchanged. On the empty input the machine goes `start` → `empty`, writes `0`, moves left onto a blank in `carry`, writes `1`, and stops with `10` on the tape, which is correct. On every nonempty input the first symbol is a digit, so the new state just hands off to `right` and the machine behaves exactly as before. (My first instinct was to add a blank rule to `right` itself, but `right` already uses the blank to detect the end of the scan, so a separate state is needed.) The lesson is to check the smallest input, here the empty string, before calling a machine finished.

Q2.1 Turing reduction warmup (a)

The statement is true: $L \leq_T \overline{L}$ for every language L.

Let O be an oracle that decides $\overline{L}$, i.e., O halts on every input and $O(x)$ accepts if and only if $x \in \overline{L}$. Consider the following machine with access to O.

Input: A string x
Output: accept if $x \in L$, reject otherwise

```
1: function M_L(x)
2:     if O(x) accepts then return reject
3:     else return accept
4:     end if
5: end function
```

M_L **always halts.** It makes a single call to O, which halts on every input because O decides $\overline{L}$, and then returns.

Correctness. By the definition of the complement, the fact that O decides $\overline{L}$, and the code of M_L,

$$x \in L \implies x \notin \overline{L} \implies O(x) \text{ rejects} \implies M_L(x) \text{ accepts},$$
$$x \notin L \implies x \in \overline{L} \implies O(x) \text{ accepts} \implies M_L(x) \text{ rejects}.$$

So M_L decides L using an oracle for $\overline{L}$, which is exactly the definition of $L \leq_T \overline{L}$. Since L was arbitrary, every language is Turing-reducible to its complement.

(This says nothing about whether L is decidable. The oracle is a hypothetical device, and all we have shown is that deciding L is no harder than deciding $\overline{L}$.)

Q2.2 Turing reduction warmup (b)

The statement is true: if L is decidable, then $L \leq_T L'$ for every language L'.

Since L is decidable, there is a Turing machine D that halts on every input and accepts exactly the strings in L. Let O be an oracle that decides L'. Consider the following machine, which has access to O but simply never uses it.

Input: A string x
Output: accept if $x \in L$, reject otherwise

1: **function** $M_L(x)$
2: **return** $D(x)$ ▷ the oracle O is never called
3: **end function**

M_L **always halts.** Its only step is to run D, which halts on every input because D is a decider.

Correctness. Because D decides L,

$$x \in L \implies D(x) \text{ accepts} \implies M_L(x) \text{ accepts},$$
$$x \notin L \implies D(x) \text{ rejects} \implies M_L(x) \text{ rejects}.$$

So M_L decides L. A Turing reduction is a Turing machine that decides L when given access to an oracle for L', and it may call the oracle any finite number of times, including zero times. Hence M_L is a valid Turing reduction and $L \leq_T L'$. Since L' was arbitrary, a decidable language is Turing-reducible to every language, in particular to $\emptyset$, to Σ^*, and to undecidable languages like L_{HALT}.

Q2.3 Turing reduction warmup (c)

The statement is false. Here is a counterexample: $L = L' = L_{\text{HALT}}$.

$L_{\text{HALT}} \leq_T L_{\text{HALT}}$. More generally, every language A satisfies $A \leq_T A$. Let O be an oracle that decides A and consider

Input: A string x
Output: accept if $x \in A$, reject otherwise
1: **function** $M_A(x)$
2: **return** $O(x)$
3: **end function**

M_A always halts because its only step is one call to O, which halts on every input. And since O decides A: $x \in A \Rightarrow O(x)$ accepts $\Rightarrow M_A(x)$ accepts, and $x \notin A \Rightarrow O(x)$ rejects $\Rightarrow M_A(x)$ rejects. So M_A decides A with an oracle for A, i.e., $A \leq_T A$. Taking $A = L_{\text{HALT}}$ gives $L_{\text{HALT}} \leq_T L_{\text{HALT}}$.

L_{HALT} is undecidable. This was proved in lecture.

So with $L = L' = L_{\text{HALT}}$ there exists a language L' with $L \leq_T L'$, yet L is undecidable. This disproves the statement.

The reason the statement fails: $L \leq_T L'$ only says that L is no harder than L'. That lets decidability flow from L' to L (if L' is decidable, so is L), and undecidability flow from L to L' (if L is undecidable, so is L'), but it gives no information about L when L' is undecidable. (Part (a) gives another counterexample: $L_{\text{HALT}} \leq_T \overline{L_{\text{HALT}}}$ with L_{HALT} undecidable.)

Q3.1 Goldbach conjecture (a)

The machine checks the even numbers $4, 6, 8, \ldots$ in increasing order. For each one it tries every way of writing it as a sum of two numbers and asks whether both are prime. If it ever finds an even number that is not a sum of two primes, it halts; otherwise it moves on to the next even number, forever.

Input: Any string w (ignored)

```
1:  function M_Search(w)
2:      n ← 4
3:      loop
4:          found ← false
5:          for p = 2, 3, ..., n − 2 do
6:              if isPrime(p) and isPrime(n − p) then
7:                  found ← true
8:              end if
9:          end for
10:         if not found then return accept          ▷ n is a counterexample: halt
11:         end if
12:         n ← n + 2
13:     end loop
14: end function
```

Each pass of the outer loop halts. For a fixed n, the inner **for** loop runs $n - 3$ times, and each iteration makes two calls to isPrime, which always returns. So every pass through the outer loop takes a finite number of steps, and the only way for M_{Search} not to halt is to go through infinitely many passes.

Each pass tests the right thing. Fix an even $n \geq 4$. If $n = p + q$ for primes p, q, then $2 \leq p \leq n - 2$ (since $q \geq 2$), so the inner loop reaches this value of p, sees that p and $n - p = q$ are both prime, and sets *found* to true. Conversely, if *found* is set to true at some p, then p and $n - p$ are primes with $p + (n - p) = n$. So after the inner loop, *found* is true if and only if n is a sum of two primes.

M_{Search} halts if and only if the Goldbach Conjecture is false.

- Suppose the conjecture is false. Then some even $n > 2$ is not a sum of two primes; let n^* be the smallest such. For every even n with $4 \leq n < n^*$, the pass for n sets *found* to true and does not halt, so the machine goes on to $n + 2$. After finitely many passes it reaches $n = n^*$, where *found* stays false, and it halts.

- Suppose the conjecture is true. Then every even $n \geq 4$ is a sum of two primes, so every pass sets *found* to true and the **return** is never executed. The outer loop runs forever, so M_{Search} does not halt.

The input w is never read, so this behavior is the same on every input.

Q3.2 Goldbach conjecture (b)

Let H be an oracle that decides L_{HALT}: H halts on every input and $H(\langle M\rangle, x)$ accepts if and only if $M(x)$ halts. By part (a), M_{SEARCH} halts (on any input, say the empty string ε) exactly when the Goldbach Conjecture is false. So asking H whether M_{SEARCH} halts on ε is the same as asking whether the conjecture is false.

Input: Any string w (ignored)
Output: accept if the Goldbach Conjecture is true, reject otherwise

```
1: function M_G(w)
2:     if H(⟨M_SEARCH⟩, ε) accepts then return reject
3:     else return accept
4:     end if
5: end function
```

The string $\langle M_{\text{SEARCH}}\rangle$ is a fixed, finite piece of code (the pseudocode from part (a)), so it can be written directly into M_G.

M_G **always halts.** It makes one call to H, which halts on every input because H decides L_{HALT}, and then returns. In particular M_G never runs M_{SEARCH} itself.

Correctness. By part (a), the definition of L_{HALT}, the fact that H decides L_{HALT}, and the code of M_G,

$$\begin{aligned}
\text{Goldbach true} &\implies M_{\text{SEARCH}}(\varepsilon) \text{ does not halt} \\
&\implies (\langle M_{\text{SEARCH}}\rangle, \varepsilon) \notin L_{\text{HALT}} \\
&\implies H(\langle M_{\text{SEARCH}}\rangle, \varepsilon) \text{ rejects} \\
&\implies M_G(w) \text{ accepts,}
\end{aligned}$$

$$\begin{aligned}
\text{Goldbach false} &\implies M_{\text{SEARCH}}(\varepsilon) \text{ halts} \\
&\implies (\langle M_{\text{SEARCH}}\rangle, \varepsilon) \in L_{\text{HALT}} \\
&\implies H(\langle M_{\text{SEARCH}}\rangle, \varepsilon) \text{ accepts} \\
&\implies M_G(w) \text{ rejects.}
\end{aligned}$$

So on every input, M_G accepts if the Goldbach Conjecture is true and rejects otherwise.

Q3.3 Goldbach conjecture (c)

Replacing H by M_{SH} gives an ordinary Turing machine M'_G (no oracle) that makes one call $M_{SH}(\langle M_{\text{SEARCH}}\rangle, \varepsilon)$ and returns the opposite answer. Whether this "works" depends on the length of that query.

If $(\langle M_{\textbf{Search}}\rangle, \varepsilon)$ **has** 1000 **or more characters**, it is not in $L_{\text{SmallHalt}}$ by definition, whatever M_{SEARCH} does. So M_{SH} rejects it and M'_G always accepts, regardless of whether the conjecture is true. In this case the solution does not work: M'_G's answer is not based on the behavior of M_{SEARCH} at all. (The correctness chain from part (b) breaks at the step "$(\langle M_{\text{SEARCH}}\rangle, \varepsilon) \in L_{\text{HALT}} \implies$ oracle accepts," which is no longer justified.)

If $(\langle M_{\textbf{Search}}\rangle, \varepsilon)$ **has fewer than** 1000 **characters**, then M_{SH} agrees with H on this particular query, so the proof from part (b) goes through word for word, and M'_G is a Turing machine with no oracle that accepts on every input if the conjecture is true and rejects on every input otherwise.

This sounds exciting, but it does not help us learn the answer. The reason is that the proof that $L_{\text{SmallHalt}}$ is decidable is non-constructive. $L_{\text{SmallHalt}}$ is finite, so it is decided by a machine that hard-codes the finite list of its members (as in HW5, Q6(a)), but we are not told which strings are on that list. Writing down M_{SH} would require knowing, for each of the finitely many pairs of length < 1000, whether M halts on x, and for the pair $(\langle M_{\text{SEARCH}}\rangle, \varepsilon)$ that is exactly the question of whether the Goldbach Conjecture is true. So M_{SH} exists, but nobody can actually write it down and run it without already having resolved the conjecture.

In fact, the existence of a Turing machine that answers the Goldbach question was never in doubt: one of the two trivial machines "always accept" and "always reject" is correct, we just do not know which. Part (b) is interesting only because the oracle H is a single machine that would answer this and countless other open questions at once, and part (c) shows that the finite restriction $L_{\text{SmallHalt}}$ does not give us anything like that.

Q4.1 Professor ϒ's reductions (a)

The error: the reduction goes in the wrong direction, so there is no contradiction.

The argument constructs $M_{\overline{\text{BARBER}}}$ that decides $\overline{L_{\text{BARBER}}}$ given a decider D_{ACC} for L_{ACC}. That is a proof of $\overline{L_{\text{BARBER}}} \leq_T L_{\text{ACC}}$: if L_{ACC} were decidable, then $\overline{L_{\text{BARBER}}}$ would be too. But L_{ACC} is undecidable, so the machine D_{ACC} does not exist, the "decider" $M_{\overline{\text{BARBER}}}$ was never actually built, and no contradiction arises. The last sentence of the proof treats "$\overline{L_{\text{BARBER}}}$ is decidable relative to L_{ACC}" as if it were "$\overline{L_{\text{BARBER}}}$ is decidable," which is not what was shown.

Reducing a language to an undecidable language proves nothing about it. By Q2(b), even a decidable language such as $\emptyset$ satisfies $\emptyset \leq_T L_{\text{ACC}}$, so this style of argument would "prove" that $\emptyset$ is undecidable. What is needed is a reduction in the other direction, $L_{\text{ACC}} \leq_T \overline{L_{\text{BARBER}}}$ (or $L_{\text{BARBER}} \leq_T \overline{L_{\text{BARBER}}}$, which follows from Q2(a) with $L = L_{\text{BARBER}}$, since L_{BARBER} is known to be undecidable). Only showing that a decider for $\overline{L_{\text{BARBER}}}$ would yield a decider for a known undecidable language gives a contradiction.

Q4.2 Professor ϒ's reductions (b)

The error: line 2 runs $M(x)$ directly, so $M_{\mathbf{ACC}}$ does not halt on every input and is not a decider.

A Turing reduction $L_{\text{ACC}} \leq_T L_{\text{LOOP}}$ must be a machine that decides L_{ACC} given the oracle, which in particular means it must halt on every input. The oracle is the only tool that is guaranteed to return. The machine being reduced from, M, is an arbitrary Turing machine, and simulating it can take forever. Exactly when $(\langle M \rangle, x) \notin L_{\text{ACC}}$ because $M(x)$ loops, the simulation on line 2 never finishes, so lines 3 to 5 are never reached and M_{ACC} loops as well.

The proof hides this in the second chain:

$$\begin{aligned}(\langle M \rangle, x) \notin L_{\text{ACC}} &\implies M(x) \text{ loops or rejects} \\ &\implies D_{\text{LOOP}}(\langle M \rangle, x) \text{ accepts or } M(x) \text{ rejects} \\ &\implies M_{\text{ACC}} \text{ rejects.}\end{aligned}$$

The last implication is false in the "loops" case: the fact that $D_{\text{LOOP}}(\langle M \rangle, x)$ would accept is irrelevant, because M_{ACC} is stuck on line 2 and never asks the oracle. So M_{ACC} does not reject these inputs, it fails to halt, and the claim "M_{ACC} decides L_{ACC}" is unproven. (The first chain is fine: if $M(x)$ accepts, the simulation finishes and line 3 accepts.)

The statement itself is true, and the fix is to consult the oracle before running anything: first ask $D_{\text{LOOP}}(\langle M \rangle, x)$; if it accepts, reject; otherwise $M(x)$ is guaranteed to halt, so it is now safe to run $M(x)$ and return its answer.

Q5.1 A powerful diagonalization (a)

Let A be countably infinite. Then its elements can be listed without repetition as $A = \{a_1, a_2, a_3, \ldots\}$. "Uncountably infinite" means infinite and not countable, so I show both.

$\mathcal{P}(A)$ **is infinite.** The singletons $\{a_1\}, \{a_2\}, \{a_3\}, \ldots$ are infinitely many distinct subsets of A, hence infinitely many distinct elements of $\mathcal{P}(A)$.

$\mathcal{P}(A)$ **is not countable.** Suppose for contradiction that $\mathcal{P}(A)$ is countable. Since it is infinite, there is an enumeration $S_1, S_2, S_3, \ldots$ of $\mathcal{P}(A)$ in which every subset of A appears (as some S_j). Picture the infinite table whose row i is the subset S_i and whose column j is the element a_j, with entry (i, j) equal to 1 if $a_j \in S_i$ and 0 otherwise:

	a_1	a_2	a_3	$\cdots$
S_1	**1**	0	1	$\cdots$
S_2	0	**0**	1	$\cdots$
S_3	1	1	**1**	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Now define the "flipped diagonal" set

$$D = \{a_i : a_i \notin S_i\} \subseteq A.$$

In the table, D is the row obtained by flipping every entry on the diagonal: $a_i \in D$ if and only if entry (i, i) is 0.

D is a subset of A, so $D \in \mathcal{P}(A)$, and therefore $D = S_j$ for some index j. Ask whether $a_j \in D$:

- If $a_j \in D$, then by the definition of D, $a_j \notin S_j = D$, a contradiction.
- If $a_j \notin D$, then $a_j \notin S_j$ (since $S_j = D$), so by the definition of D, $a_j \in D$, a contradiction.

Either way we reach a contradiction, so no such j exists: D differs from every S_i (precisely in whether it contains a_i). This contradicts the assumption that the list $S_1, S_2, \ldots$ contains every subset of A. Hence $\mathcal{P}(A)$ is not countable.

Together, $\mathcal{P}(A)$ is uncountably infinite.

Q5.2 A powerful diagonalization (b)

Let $L \subseteq \Sigma^*$ be an infinite language. The subsets of L are exactly the languages $L' \subseteq L$, so the claim is that not every element of $\mathcal{P}(L)$ is decidable. The plan: $\mathcal{P}(L)$ is uncountable by part (a), but there are only countably many decidable languages, so most subsets of L are undecidable.

Step 1: L is countably infinite. Σ is finite, so Σ^* is countable: list its strings by length, and within each length in alphabetical order. Every string has some finite length ℓ, and there are only finitely many strings of length at most ℓ, so every string appears at a finite position in this list. Now go through this list and delete every string that is not in L. What remains is a list $\ell_1, \ell_2, \ell_3, \ldots$ that contains every string of L exactly once, and it is infinite because L is infinite. So L is countably infinite.

Step 2: $\mathcal{P}(L)$ is uncountable. This is part (a) applied to $A = L$.

Step 3: there are only countably many decidable languages. Every Turing machine M has a finite description $\langle M \rangle$, a string over a fixed finite alphabet, and different machines have different descriptions. By the argument of Step 1 (with L replaced by the set of valid encodings), the set of all Turing machines is countable; this was also shown in lecture. Now map each decidable language L' to the encoding $\langle M_{L'} \rangle$ of some fixed Turing machine that decides it (one exists by the definition of decidability; if there are several, pick the first one in the list from Step 1). Two different languages cannot be mapped to the same machine, because a single machine decides only one language. So this map is an injection from the set of decidable languages into a countable set, and therefore the set of decidable languages is countable.

Step 4: conclusion. Suppose for contradiction that every subset $L' \subseteq L$ were decidable. Then $\mathcal{P}(L)$ would be a subset of the set of decidable languages. A subset of a countable set is countable (list the big set and delete the elements not in the subset, as in Step 1), so $\mathcal{P}(L)$ would be countable, contradicting Step 2. Hence some subset $L' \subseteq L$ is undecidable.

(In fact the argument shows much more: all but countably many of the uncountably many subsets of L are undecidable.)

Q6.1 Deciding Undecidability (a)

$L = \{\langle M, x\rangle : M(x) \text{ halts within } 2^{|x|} \text{ steps}\}$ is **decidable**.

The point is that the number of steps to check is a finite number that can be computed from the input, so we can simulate M for exactly that long and then stop, whatever M does afterwards.

Input: $\langle M, x\rangle$, where M is a Turing machine and x a string
Output: accept if $M(x)$ halts within $2^{|x|}$ steps, reject otherwise

1: **function** $D_L(\langle M, x\rangle)$
2: $T \leftarrow 2^{|x|}$ ▷ compute $|x|$ by scanning x, then double a counter $|x|$ times
3: Simulate M on input x, one step at a time, keeping a count of the steps simulated
4: **if** M enters a halting state (accept or reject) after at most T steps **then return** accept
5: **end if**
6: **if** T steps have been simulated and M has not halted **then** stop the simulation and **return** reject
7: **end if**
8: **end function**

D_L **always halts.** Computing T takes finitely many steps. The simulation runs for at most T steps of M, and simulating a single step of M (as a universal Turing machine does: look up the transition for the current state and symbol, update the tape) takes a finite amount of time. So the simulation stops after at most T simulated steps, either because M halted or because the counter reached T, and D_L returns. D_L never runs M without a bound.

Correctness. The simulation is exact, so M halts within the first T simulated steps if and only if $M(x)$ really halts within $T = 2^{|x|}$ steps.

- If $\langle M, x\rangle \in L$, then $M(x)$ halts within $2^{|x|}$ steps, so the simulation sees M halt at or before step T, and D_L accepts.
- If $\langle M, x\rangle \notin L$, then $M(x)$ has not halted after $2^{|x|}$ steps (it halts later or never), so the simulation reaches the step bound T without seeing a halting state, and D_L rejects.

So D_L decides L, and L is decidable. (Unlike L_{HALT}, this language only asks about a bounded prefix of the computation, which is why it is decidable.)

Q6.2 Deciding Undecidability (b)

$L_{\text{SELF-LOOP}}$ is **undecidable**. Recall $L_{\text{SELF-LOOP}} = \{\langle M\rangle : M \text{ loops on input } \langle M\rangle\}$. I show $L_{\text{HALT}} \leq_T L_{\text{SELF-LOOP}}$; since L_{HALT} is undecidable (lecture), so is $L_{\text{SELF-LOOP}}$.

Let O be an oracle that decides $L_{\text{SELF-LOOP}}$: O halts on every input, and $O(\langle N\rangle)$ accepts if and only if $N(\langle N\rangle)$ loops. The idea is to turn the question "does M halt on x?" into a question about a machine M' that behaves the same way on every input, in particular on its own code, as M does on x.

Input: $(\langle M\rangle, x)$, where M is a Turing machine and x a string
Output: accept if $M(x)$ halts, reject otherwise

1: **function** $M_{\text{HALT}}(\langle M\rangle, x)$
2: Construct the code $\langle M'\rangle$ of the following machine M':
function $M'(w)$: ▷ w is ignored
Run $M(x)$ ▷ x and $\langle M\rangle$ are hard-coded into M'
return accept
3: **if** $O(\langle M'\rangle)$ accepts **then return** reject
4: **else return** accept
5: **end if**
6: **end function**

M_{HALT} always halts. Constructing $\langle M'\rangle$ is just string manipulation: M_{HALT} writes down a fixed template of code with the strings $\langle M\rangle$ and x pasted in. It does not run M or M'. Then it makes one call to O, which halts on every input because O decides $L_{\text{SELF-LOOP}}$.

Key property of M'. For every input w, $M'(w)$ ignores w and runs $M(x)$. If $M(x)$ halts (accepting or rejecting), M' then accepts, so $M'(w)$ halts. If $M(x)$ loops, $M'(w)$ never gets past the simulation, so $M'(w)$ loops. In particular, taking $w = \langle M'\rangle$:

$$M'(\langle M'\rangle) \text{ loops} \iff M(x) \text{ loops}.$$

Correctness. By the definition of L_{HALT}, the key property, the definition of $L_{\text{SELF-LOOP}}$, the fact that O decides $L_{\text{SELF-LOOP}}$, and the code of M_{HALT},

$$\begin{aligned}
(\langle M\rangle, x) \in L_{\text{HALT}} &\implies M(x) \text{ halts} \\
&\implies M'(\langle M'\rangle) \text{ halts} \\
&\implies \langle M'\rangle \notin L_{\text{SELF-LOOP}} \\
&\implies O(\langle M'\rangle) \text{ rejects} \\
&\implies M_{\text{HALT}} \text{ accepts},
\end{aligned}$$

$$\begin{aligned}
(\langle M\rangle, x) \notin L_{\text{HALT}} &\implies M(x) \text{ loops} \\
&\implies M'(\langle M'\rangle) \text{ loops} \\
&\implies \langle M'\rangle \in L_{\text{SELF-LOOP}} \\
&\implies O(\langle M'\rangle) \text{ accepts} \\
&\implies M_{\text{HALT}} \text{ rejects}.
\end{aligned}$$

So M_{HALT} decides L_{HALT} using an oracle for $L_{\text{SELF-LOOP}}$, that is, $L_{\text{HALT}} \leq_T L_{\text{SELF-LOOP}}$. If $L_{\text{SELF-LOOP}}$ were decidable, we could replace O by its decider and obtain a decider for L_{HALT}, which does not exist. Therefore $L_{\text{SELF-LOOP}}$ is undecidable.

Q6.3 Deciding Undecidability (c)

$L_{\text{AND-LOOP}}$ is **undecidable**. Recall

$$L_{\text{AND-LOOP}} = \{(\langle M\rangle, x, y) : M \text{ loops on } x \text{ and loops on } y\}.$$

I show $L_{\text{HALT}} \leq_T L_{\text{AND-LOOP}}$; since L_{HALT} is undecidable, so is $L_{\text{AND-LOOP}}$.

Let O be an oracle that decides $L_{\text{AND-LOOP}}$: O halts on every input, and $O(\langle M\rangle, x, y)$ accepts if and only if M loops on x and M loops on y. The trick is that nothing stops us from asking about the same string twice: "M loops on x and M loops on x" is just "M loops on x," which is the opposite of "M halts on x."

Input: $(\langle M\rangle, x)$, where M is a Turing machine and x a string
Output: accept if $M(x)$ halts, reject otherwise

```
1: function M_HALT(⟨M⟩, x)
2:     if O(⟨M⟩, x, x) accepts then return reject
3:     else return accept
4:     end if
5: end function
```

M_{HALT} always halts. It copies x once to form the query $(\langle M\rangle, x, x)$, makes a single call to O, which halts on every input because O decides $L_{\text{AND-LOOP}}$, and returns. It never runs M.

Correctness. By the definition of $L_{\text{AND-LOOP}}$ with $y = x$, the condition "M loops on x and M loops on x" holds exactly when $M(x)$ loops. Hence, using the definition of L_{HALT}, the fact that O decides $L_{\text{AND-LOOP}}$, and the code of M_{HALT},

$$\begin{aligned}
(\langle M\rangle, x) \in L_{\text{HALT}} &\implies M(x) \text{ halts} \\
&\implies (\langle M\rangle, x, x) \notin L_{\text{AND-LOOP}} \\
&\implies O(\langle M\rangle, x, x) \text{ rejects} \\
&\implies M_{\text{HALT}} \text{ accepts,}
\end{aligned}$$

$$\begin{aligned}
(\langle M\rangle, x) \notin L_{\text{HALT}} &\implies M(x) \text{ loops} \\
&\implies (\langle M\rangle, x, x) \in L_{\text{AND-LOOP}} \\
&\implies O(\langle M\rangle, x, x) \text{ accepts} \\
&\implies M_{\text{HALT}} \text{ rejects.}
\end{aligned}$$

So M_{HALT} decides L_{HALT} with an oracle for $L_{\text{AND-LOOP}}$, i.e., $L_{\text{HALT}} \leq_T L_{\text{AND-LOOP}}$. A decider for $L_{\text{AND-LOOP}}$ would therefore give a decider for L_{HALT}, which does not exist, so $L_{\text{AND-LOOP}}$ is undecidable.

(If one prefers not to repeat the input, the same proof works with the query $O(\langle M'\rangle, x, y)$ for any two strings $x \neq y$, where M' is the machine from part (b) that ignores its input and runs $M(x)$: then M' loops on both x and y if and only if $M(x)$ loops.)

Q7 Halting on Prefixes

Let O be an oracle that decides $L_{\text{PREFIX-HALT}}$: O halts on every input, and $O(\langle M\rangle, z, k)$ accepts if and only if M halts on exactly k of the prefixes of z.

Idea. For a string z, write $h(z)$ for the number of prefixes of z on which M halts. A string of length ℓ has $\ell + 1$ prefixes, so $h(z)$ is a well-defined integer in $\{0, 1, \ldots, |z| + 1\}$, and exactly one of the queries $(\langle M\rangle, z, k)$ for $k = 0, \ldots, |z| + 1$ is in $L_{\text{PREFIX-HALT}}$, namely the one with $k = h(z)$. So by calling the oracle at most $|z| + 2$ times, we can compute $h(z)$. Now let x' be x with its last symbol removed. The prefixes of x are the prefixes of x' together with x itself (and x is not a prefix of x', being longer), so

$$h(x) = h(x') + \begin{cases} 1 & \text{if } M(x) \text{ halts,} \\ 0 & \text{if } M(x) \text{ loops.} \end{cases}$$

Comparing $h(x)$ with $h(x')$ therefore tells us whether M halts on x, without ever running M. The empty string has only itself as a prefix, so for $x = \varepsilon$ we simply have: $M(\varepsilon)$ halts if and only if $h(\varepsilon) = 1$.

Input: $(\langle M\rangle, x)$, where M is a Turing machine and x a string
Output: accept if $M(x)$ halts, reject otherwise

```
1: function CountHalt(⟨M⟩, z)                          ▷ returns h(z), using the oracle
2:     for k = 0, 1, ..., |z| + 1 do
3:         if O(⟨M⟩, z, k) accepts then return k
4:         end if
5:     end for
6: end function

7: function M_HALT(⟨M⟩, x)
8:     if x = ε then
9:         if O(⟨M⟩, ε, 1) accepts then return accept else return reject
10:        end if
11:    end if
12:    x' ← x with its last symbol removed
13:    c ← CountHalt(⟨M⟩, x)
14:    c' ← CountHalt(⟨M⟩, x')
15:    if c = c' + 1 then return accept
16:    else return reject
17:    end if
18: end function
```

M_{HALT} always halts. The only operations are forming strings (removing the last symbol of x, writing down the triples $(\langle M\rangle, z, k)$), comparing small integers, and calling O. Each call to O halts because O decides $L_{\text{PREFIX-HALT}}$. CountHalt makes at most $|z| + 2$ calls, so M_{HALT} makes at most $(|x| + 2) + (|x| + 1) = 2|x| + 3$ oracle calls in total, all of which return. M is never run.

CountHalt returns $h(z)$. By the definition of $L_{\text{PREFIX-HALT}}$ and the fact that O decides it, $O(\langle M\rangle, z, k)$ accepts if and only if $k = h(z)$. Since $h(z) \in \{0, \ldots, |z| + 1\}$, the loop reaches $k = h(z)$ (all earlier k are rejected) and returns it.

Correctness. First let $x = \varepsilon$. Its only prefix is ε, so $h(\varepsilon) = 1$ if $M(\varepsilon)$ halts and $h(\varepsilon) = 0$

otherwise. Hence

$$\begin{aligned}(\langle M\rangle,\varepsilon)\in L_{\text{HALT}} &\Longrightarrow h(\varepsilon)=1\\ &\Longrightarrow (\langle M\rangle,\varepsilon,1)\in L_{\text{PREFIX-HALT}}\\ &\Longrightarrow O(\langle M\rangle,\varepsilon,1)\text{ accepts}\\ &\Longrightarrow M_{\text{HALT}}\text{ accepts,}\end{aligned}$$

$$\begin{aligned}(\langle M\rangle,\varepsilon)\notin L_{\text{HALT}} &\Longrightarrow h(\varepsilon)=0\\ &\Longrightarrow (\langle M\rangle,\varepsilon,1)\notin L_{\text{PREFIX-HALT}}\\ &\Longrightarrow O(\langle M\rangle,\varepsilon,1)\text{ rejects}\\ &\Longrightarrow M_{\text{HALT}}\text{ rejects.}\end{aligned}$$

Now let $x\neq\varepsilon$, so $x=x'a$ for a symbol a. The prefixes of x are exactly the prefixes of x' (those of length $\leq|x'|$) plus the one extra prefix x (of length $|x|$). By the previous paragraph, the two calls to CountHalt give $c=h(x)$ and $c'=h(x')$. So

$$\begin{aligned}(\langle M\rangle,x)\in L_{\text{HALT}} &\Longrightarrow M(x)\text{ halts}\\ &\Longrightarrow h(x)=h(x')+1\\ &\Longrightarrow c=c'+1\\ &\Longrightarrow M_{\text{HALT}}\text{ accepts,}\end{aligned}$$

$$\begin{aligned}(\langle M\rangle,x)\notin L_{\text{HALT}} &\Longrightarrow M(x)\text{ loops}\\ &\Longrightarrow h(x)=h(x')\\ &\Longrightarrow c\neq c'+1\\ &\Longrightarrow M_{\text{HALT}}\text{ rejects.}\end{aligned}$$

So M_{HALT} decides L_{HALT} using an oracle for $L_{\text{PREFIX-HALT}}$, i.e., $L_{\text{HALT}}\leq_T L_{\text{PREFIX-HALT}}$. Since L_{HALT} is undecidable, a decider for $L_{\text{PREFIX-HALT}}$ cannot exist (it would give a decider for L_{HALT}), so $L_{\text{PREFIX-HALT}}$ is undecidable.

Another way to do it. The counting can be avoided by building a new machine: let M' be the machine that, on input w, accepts immediately if $w\neq x$, and otherwise runs $M(x)$ and then accepts. M' halts on all $|x|$ proper prefixes of x (they differ from x), and halts on the prefix x itself if and only if $M(x)$ halts. So M' halts on exactly $|x|+1$ prefixes of x if and only if $M(x)$ halts, and the single query $O(\langle M'\rangle,x,|x|+1)$ decides L_{HALT}.

Q8 We have a common language! (optional extra credit)

Let $A, B \subseteq \{0,1\}^*$ be arbitrary languages. Define

$$C = \{0x : x \in A\} \cup \{1x : x \in B\} \subseteq \{0,1\}^*.$$

That is, C stores A and B side by side, tagged by their first bit: a string of C starting with 0 is a member of A with a 0 in front, and a string of C starting with 1 is a member of B with a 1 in front. (The empty string is not in C.)

Membership in C is easy to read off. For every $x \in \{0,1\}^*$: $0x \in C$ if and only if $x \in A$. Indeed, if $x \in A$ then $0x \in C$ by definition. Conversely, if $0x \in C$, then $0x$ starts with 0, so it cannot be of the form $1y$; hence $0x = 0x'$ for some $x' \in A$, and stripping the first bit gives $x = x' \in A$. In the same way, $1x \in C$ if and only if $x \in B$.

$A \leq_T C$. Let O be an oracle that decides C.

Input: A string $x \in \{0,1\}^*$
Output: accept if $x \in A$, reject otherwise

```
1: function M_A(x)
2:     return O(0x)
3: end function
```

M_A always halts: it prepends one bit to x and makes one call to O, which halts on every input because O decides C. By the property above and the fact that O decides C,

$$x \in A \Longrightarrow 0x \in C \Longrightarrow O(0x) \text{ accepts} \Longrightarrow M_A \text{ accepts},$$
$$x \notin A \Longrightarrow 0x \notin C \Longrightarrow O(0x) \text{ rejects} \Longrightarrow M_A \text{ rejects}.$$

So M_A decides A with an oracle for C, i.e., $A \leq_T C$.

$B \leq_T C$. Symmetrically, the machine $M_B(x)$ that returns $O(1x)$ always halts, and

$$x \in B \Longrightarrow 1x \in C \Longrightarrow O(1x) \text{ accepts} \Longrightarrow M_B \text{ accepts},$$
$$x \notin B \Longrightarrow 1x \notin C \Longrightarrow O(1x) \text{ rejects} \Longrightarrow M_B \text{ rejects}.$$

So $B \leq_T C$.

Hence for any two languages A and B over $\{0,1\}$ there is a language C over $\{0,1\}$ with $A \leq_T C$ and $B \leq_T C$. (Note that A and B may both be undecidable, and C is then undecidable too, which is fine: a Turing reduction only compares difficulty, it does not require any of the languages to be decidable.)